Prof. Dr. Peter Bastian, Nils Friess
IWR, Universität Heidelberg

Submission date: 29. June 2026

Exercise 1 *Update the hasc-code Repository* **(0 points)**

We regularly update the `hasc-code` repository with new code examples, fixes, and improvements. Before starting to work on the exercises, make sure you have the latest version of the repository.

To update your local copy while preserving any local changes you might have made, use the following commands:

```
git stash
git pull
git stash pop
```

The `git stash` command temporarily saves your local changes, `git pull` fetches and merges the latest updates from the remote repository, and `git stash pop` reapplies your saved changes.

If you haven't made any local changes, a simple `git pull` is sufficient.

Exercise 2 *Distributed Jacobi with MPI (and MPI+X)* **(1+4+2+3+*2+1 points)**

In the previous sheet you parallelized the Jacobi iteration on a shared-memory machine, decomposing the $n \times n$ lattice into contiguous row strips (one per thread); the threads simply read the border rows of their neighbours through *shared memory*.

We now keep the same decomposition but move to *distributed memory* using MPI. Each strip lives in a separate process, so a process can no longer read its neighbours' rows directly: they must be *communicated explicitly*. The standard technique is to surround each strip with a layer of *ghost rows* (or *halo*, see Figure 1) holding a copy of the neighbour's border row.

Recall that one Jacobi update replaces every interior value by the average of its four neighbours,

$$u^k_{i_0,i_1} = \tfrac{1}{4}\big(u^{k-1}_{i_0-1,i_1} + u^{k-1}_{i_0+1,i_1} + u^{k-1}_{i_0,i_1-1} + u^{k-1}_{i_0,i_1+1}\big), \qquad 1 \le i_0, i_1 \le n-2,$$

reading only the values of the previous iteration $k-1$.

You will need an MPI implementation such as OpenMPI. We provide a skeleton `jacobi_mpi.cc` that already sets up the MPI environment and contains the local update kernel; you fill in the data decomposition, the (empty) halo exchange and the distributed defect norm. Build it and run it with

```
mpirun -np <number_of_processes> ./jacobi_mpi <n> <iterations> [further args]
```

(a) **Getting started with MPI.** The skeleton already initializes the MPI environment (`MPI_Init`), queries the number of processes and the rank (`MPI_Comm_size`, `MPI_Comm_rank`) and finalizes cleanly (`MPI_Finalize`). Make sure you can build and start the program with `mpirun` and have each process print its rank.

Note: depending on the number of cores and your MPI installation it may be necessary to pass the option `--oversubscribe` to `mpirun` in order to start more processes than there are available cores, e.g.

```
mpirun --oversubscribe -np <number_of_processes> ./jacobi_mpi <n> <iterations>
```

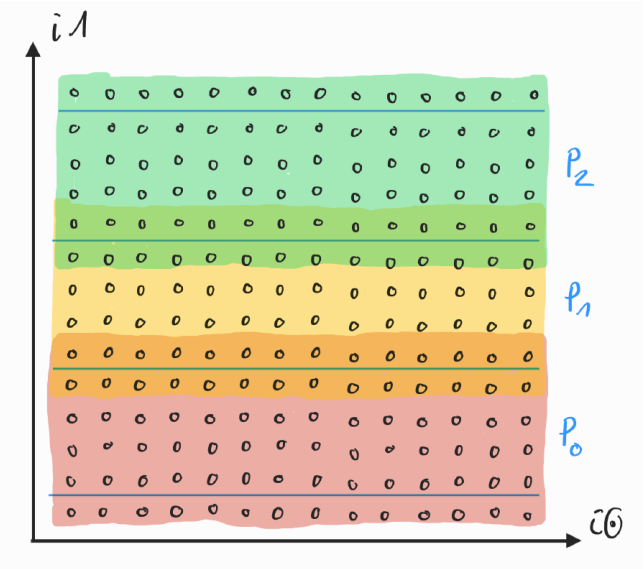

Figure 1: One-dimensional decomposition of the lattice interior into contiguous row strips, one per process (P_0, P_1, P_2). With MPI, process p additionally allocates a ghost row *above and below its strip; to update its rows in iteration k these must hold the neighbours' border values from iteration $k-1$, obtained by message passing.*

(b) **Data decomposition and halo exchange.** Partition the $n-2$ interior rows into P (roughly) equal contiguous strips and give one strip to each process. Each process allocates only its own strip *plus one ghost row at the top and one at the bottom*; it does *not* store the whole lattice. As in the previous sheet, use two buffers that are swapped after each iteration (the skeleton's update kernel already does this).

Implement one iteration as follows:

(i) **Exchange ghost rows.** Send your topmost interior row to the process above ($p-1$) and your bottommost interior row to the process below ($p+1$), and receive their border rows into your ghost rows. The first and last process have only one neighbour, so their outer ghost row stays fixed at the global boundary value.

Be careful here: if every process first calls a blocking `MPI_Send` and only afterwards `MPI_Recv`, the program may deadlock for large messages. Order the sends and receives so that the exchange is always safe (e.g. by letting even and odd ranks send and receive in the opposite order). Alternatively, you may use non-blocking sends/receives (see also the next exercise) or the `MPI_Sendrecv` function. Fill in the empty `halo_exchange` stub for this.

(ii) **Update.** Apply the Jacobi update to all interior rows of your strip, reading the ghost rows where needed (already done by the provided kernel).

As in the previous sheet, run a *fixed* number of iterations. Verify that the result agrees with the sequential reference (e.g. let rank 0 gather the strips with `MPI_Gather` and compare). Then measure the speedup over the sequential version for a few process counts and comment on what you observe.

(c) **Distributed residual.** To monitor the progress of the iteration we want to compute the norm of the defect $\|b - Az^k\|_2$. The sequential code (`jacobi_seq.cc`) does this in a *matrix-free* way: it never forms A or b but evaluates the defect directly as

$$d_{i_0,i_1} = 4\,u_{i_0,i_1} - \big(u_{i_0-1,i_1} + u_{i_0+1,i_1} + u_{i_0,i_1-1} + u_{i_0,i_1+1}\big),$$

which works because the right-hand side vanishes in the interior (only the boundary values drive the solution), so $b - Az^k = -Az^k$ there.

Implement this defect norm for the distributed strips by filling in the provided `defect_norm` stub. Two things need care:

(i) Like an update, the defect of the top and bottom rows of a strip reads the neighbours' border rows, so the *ghost rows must be up to date* before you evaluate it.

(ii) Each process computes only the local sum of squares $\sum d_{i_0,i_1}^2$ over its strip. Combine these with `MPI_Allreduce` (sum) and take the square root *afterwards*, on the global sum — summing per-process norms would be wrong.

Verify that the distributed norm matches the sequential one. You may optionally use it as a real stopping criterion (e.g. checking it every m iterations).

(d) **Overlapping communication and computation.** With the blocking exchange from (b) each process is idle while the ghost rows travel over the network. We can hide this latency: the *inner* rows of a strip do not depend on the ghost rows, so they can be updated while the halo exchange is still in flight.

If you did not already use non-blocking communication in (b), switch the exchange to `MPI_Isend` / `MPI_Irecv` now and reorganize one iteration as follows:

(i) Post the receives and sends for the ghost rows.

(ii) Update all interior rows that do *not* touch a ghost row.

(iii) Complete the communication with `MPI_Waitall`.

(iv) Update the (at most two) remaining rows that border the ghost rows.

Verify correctness and compare the performance to the blocking version from (b).

(e) **Bonus: MPI+X.** So far each process runs on a single core. To use all cores of a node we combine the two levels of parallelism: MPI handles the communication *between* processes as before, while *within* each process the strip update is parallelized over threads (reuse your OpenMP / C++ threads kernel from the previous sheet).

Implement *one* of the following two variants:

(i) **Without overlap.** One iteration becomes: the main thread performs the (blocking) halo exchange, and afterwards the whole thread team updates all interior rows of the strip.

(ii) **With overlap.** Build on the non-blocking exchange from (d): the main thread posts the halo communication, the thread team updates the inner rows while it is in flight, and after `MPI_Waitall` the team updates the remaining boundary rows.

In both variants only the main thread ever calls MPI (the communication sits outside the parallel region), so you must initialize MPI with `MPI_Init_thread` instead of `MPI_Init`, and `MPI_THREAD_FUNNELED` is sufficient. Request that level and check the level actually provided, and state in your solution which level you requested and why.

Run the hybrid code with a few combinations of processes and threads.

(f) **Comparison.** For a fixed total number of cores compare the pure-MPI solver from this sheet (one process per core) with the pure-threads version from the previous sheet. Discuss the results, e.g. in terms of the communicated volume and the surface-to-volume ratio of the strips. If you did the bonus, also include a hybrid configuration (fewer processes, several threads each) in the comparison.